

EEN245 A1

Vansh Agrawal Ayush Ahuja

January 2026

1 1. Node Embeddings with PyG

1.1 Part A: PyG basics

The code is attached in Listing 1 in the appendix.

- Number of nodes: 34.
- Number of edges: 156.
- Shape of `data.x`: 34x34.
- Shape of `data.edge_index`: 2x154.
- Number of node labels: 4.

`data.x` is a matrix of number of nodes times the number of features. Each row corresponds to the values of that node for each feature. `data.edge_index` is a 2 by number of edges sized matrix. The first row of the *i*th column represents the start point of the *i*th edge and the second row of the *i*th column represents the end point of the *i*th edge. `data.y` is the actual value which we want to train on. It can be for edges or nodes or the entire graph. We hide a part of `data.y` to be able to test our model and train it on the remaining part.

1.2 Part B: Visualizing the graph

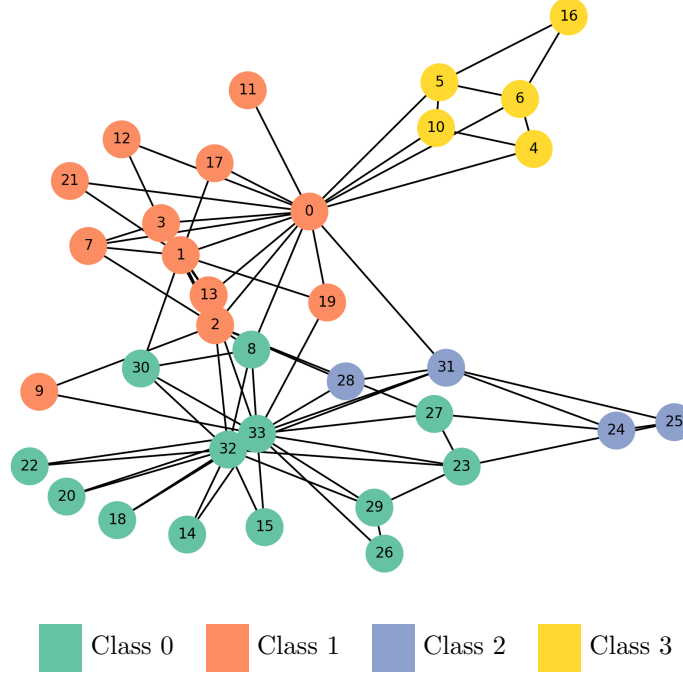


Figure 1: Visualized graph: each color stands for a class.

1.3 Part C: Structural feature embeddings

1.3.1 C1. Structural features

We compute six structural features per node: degree, betweenness centrality, closeness centrality, PageRank, eigenvector centrality, and clustering coefficient. For each feature we build a vector ordered by node index (0 to $N - 1$), then stack these vectors column-wise to obtain $X_{\text{struct}} \in \mathbb{R}^{N \times 6}$. Finally, we normalize each feature with z-score normalization (subtract mean and divide by standard deviation) to place all features on a comparable scale. The implementation is included in Listing 3 in the appendix.

1.3.2 C2. Visualize structural embeddings

We take the normalized structural feature matrix from C1 and reduce it to 2D using PCA. The 2D points are then plotted and colored by the node labels. The visualization is shown in Figure 2 and the code is in Listing 4.

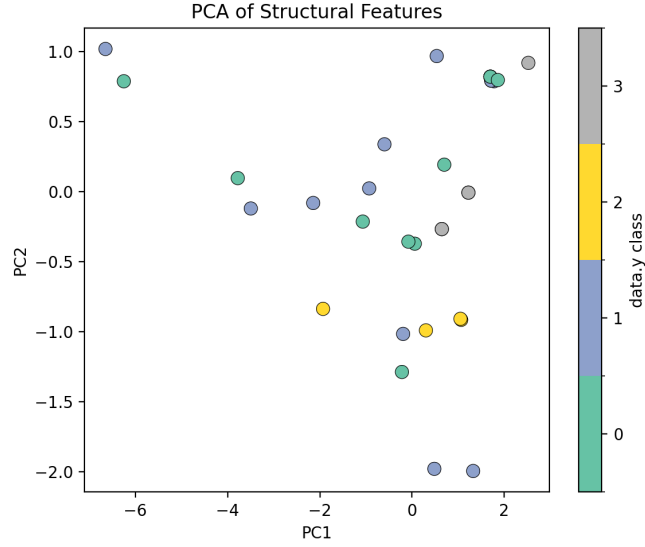


Figure 2: PCA of structural features; colors indicate the node labels.

1.3.3 C3. Node classification using structural features

We train two simple classifiers on X_{struct} (6 structural features per node): a 2-layer MLP ($6 \rightarrow 16 \rightarrow 4$) and logistic regression. We evaluate both using stratified 5-fold cross-validation (seed 42). The mean test accuracies are:

- **MLP: 56.19% $\pm$ 17.41%** (mean train accuracy 100%).
- **Logistic Regression: 44.29% $\pm$ 9.48%** (mean train accuracy 79.42%).

The implementation is shown in Listing 8.

1.4 Part D: Node classification using node features

1.4.1 D1. Node classification using node features only

We train the same two classifiers (MLP and logistic regression) on the raw node features $X = \text{data.x}$ and evaluate them with stratified 5-fold cross-validation (seed 42). The mean test accuracies are:

- **MLP: 35.71% $\pm$ 9.04%** (mean train accuracy 100%).
- **Logistic Regression: 29.52% $\pm$ 1.90%** (mean train accuracy 100%).

The implementation is shown in Listing 9.

1.4.2 D2. Visualize node features

We reduce the node features to 2D using PCA and color by node labels. The visualization is shown in Figure 3 and the code is provided in Listing 10.

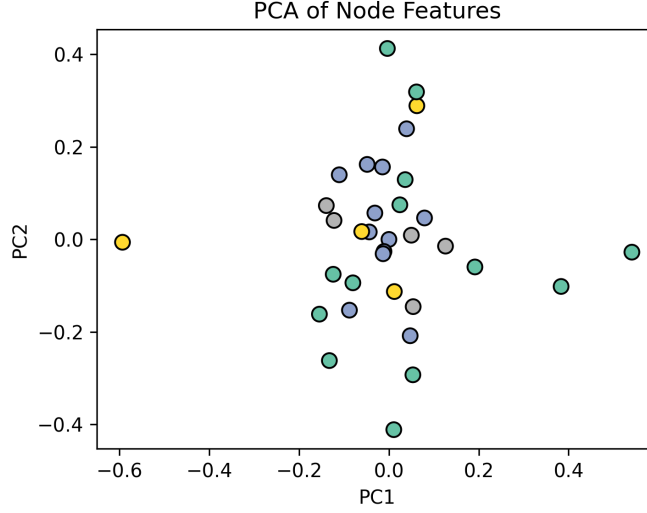


Figure 3: PCA of node features; colors indicate the node labels.

1.5 Part E: Combining structure and node features

We concatenate the structural features from Part C with the node features to form $X_{\text{comb}} = [X_{\text{struct}} \ X_{\text{attr}}]$. We train the same two classifiers (MLP and logistic regression) on X_{comb} and evaluate them with stratified 5-fold cross-validation (seed 42). The mean test accuracies are:

- **MLP: 38.10% $\pm$ 14.13%** (mean train accuracy 100%).
- **Logistic Regression: 40.95% $\pm$ 13.33%** (mean train accuracy 100%).

The implementation is shown in Listing 11.

1.6 Part F: Cora node classification (Planetoid)

We repeat the node label prediction on the Cora dataset using the official PyG masks (`train_mask`, `val_mask`, `test_mask`). We use the same structural features as in Part C (6 structural statistics), the raw node features $X_{\text{attr}} = \text{data.x}$, and their concatenation $X_{\text{comb}} = [X_{\text{struct}} \ X_{\text{attr}}]$. A small MLP (input $\rightarrow 64 \rightarrow 7$) with ReLU and dropout is trained with Adam (lr=0.01, weight decay 5×10^{-4}) and early stopping on validation accuracy. The best validation and corresponding test accuracies are:

Input features	Val acc	Test acc
X_{struct}	27.0%	27.7%
X_{attr}	61.2%	57.3%
X_{comb}	59.2%	56.9%

Table 1: Cora node classification using official Planetoid splits.

The implementation is shown in Listing 12.

1.7 Part G: Short discussion

1. **Best input.** For Karate Club, structural features performed best because $X = \text{data.x}$ is an identity matrix and does not carry label-relevant content, leading to low accuracy with node features only. For Cora, the raw node features (X_{attr}) performed best, since the bag-of-words attributes are strongly aligned with the topic labels.
2. **Why combining helps or not.** On Karate Club, combining did worse than structural-only because the identity features add noise and dilute the structural signal. On Cora, combining did not help because the structural features were relatively weak and added noise on top of already strong node attributes.
3. **What structure captures.** Structural features capture graph-topology information such as degree, centrality, clustering, and neighborhood connectivity patterns, i.e., how a node is positioned in the network. This relational role information is not present in raw identity features (Karate) and is complementary, but not necessarily aligned, with bag-of-words attributes (Cora).

2 2. Node2Vec vs Struct2Vec on Geometric-Shapes

2.1 Part A: Load graphs and visualize the original geometry

We load the GeometricShapes dataset with `FaceToEdge` to populate `edge_index`, select three graphs (indices 0, 1, 2), and plot their original geometry using `data.pos` with edges drawn from `data.edge_index`.

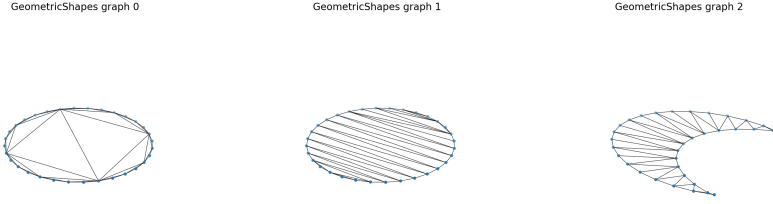


Figure 4: Original geometry for GeometricShapes graphs 0, 1, and 2 (left to right).

2.2 Part B: Node2Vec embeddings and visualization

2.2.1 B1. Implement biased random walks

We implement the Node2Vec transition rule on an undirected graph and use $p = 10$. For BFS-like and DFS-like behavior, we vary q and report both values in the results.

2.2.2 B2. Generate training walks

For each node, we generate multiple walks with a fixed length of 20 (10 walks per node in our runs) to build the skip-gram training corpus.

2.2.3 B3. Learn embeddings

We train a skip-gram model with negative sampling to produce 2D embeddings ($d = 2$). We run Node2Vec with two settings: BFS-like walks ($q = 0.5$) and DFS-like walks ($q = 5$), then plot the graph using the learned embedding coordinates and the same edges.

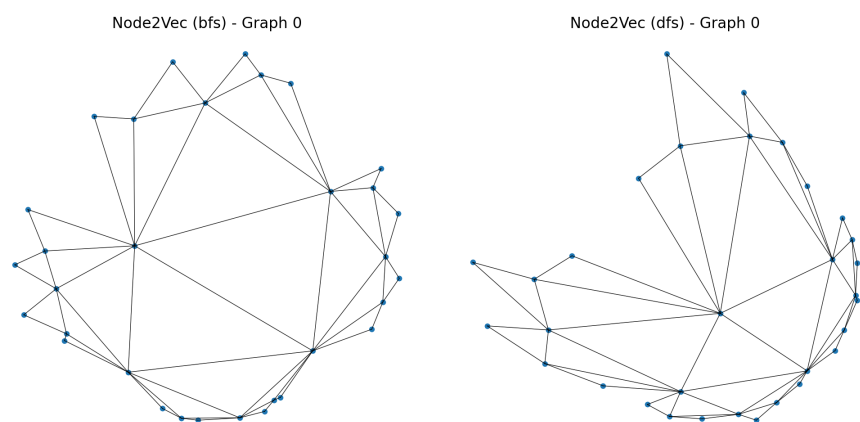


Figure 5: Node2Vec embeddings for graph 0 (left: BFS-like, right: DFS-like).

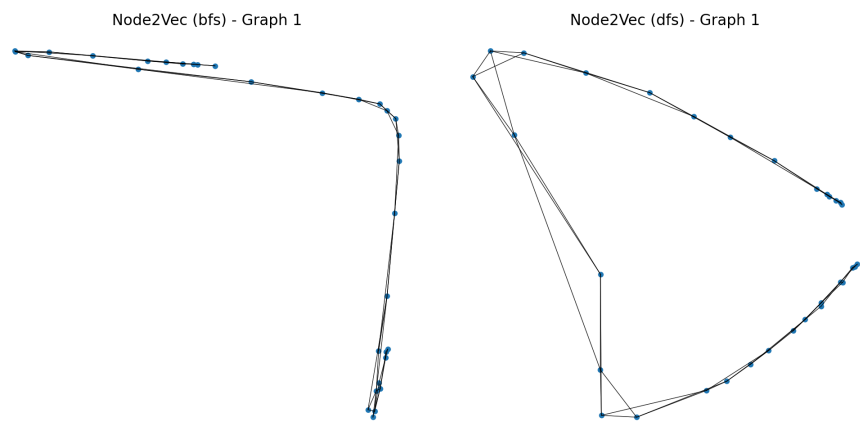


Figure 6: Node2Vec embeddings for graph 1 (left: BFS-like, right: DFS-like).

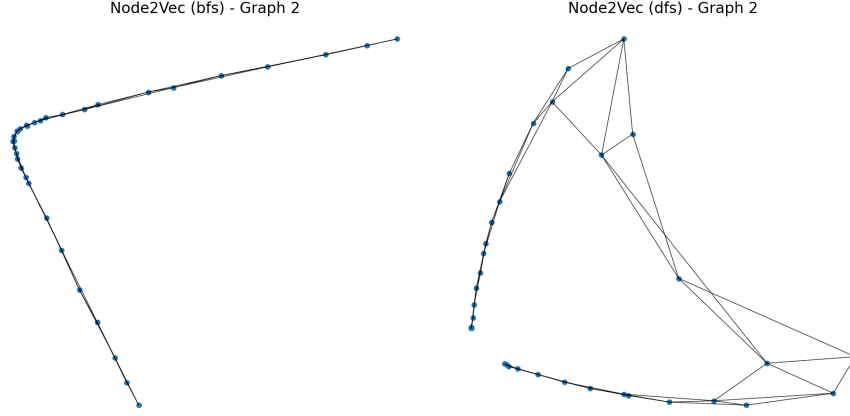


Figure 7: Node2Vec embeddings for graph 2 (left: BFS-like, right: DFS-like).

2.3 Part C: Struct2Vec embeddings and visualization (fixed $k=3$)

2.3.1 C1. Structural similarity function $g_k(u, v)$

We fix $k = 3$ and define each node's structural signature $S_k(u)$ as the multiset of degrees of nodes within k hops. For computing $g_k(u, v)$ we sort the degree sequences and compare them element-wise; if the sequence lengths differ, we pad the shorter one with zeros before taking the mean absolute difference.

2.3.2 C2. Construct the Struct2Vec graph

We construct a weighted similarity graph where each node corresponds to an original node and the edge weight is $g_k(u, v)$. To keep the graph sparse and efficient, we connect each node to its top- m most similar nodes (we use $m = 10$).

2.3.3 C3. Random walks on the Struct2Vec graph

We perform weighted random walks on this similarity graph with transition probabilities proportional to edge weights; walks are not restricted to original neighbors.

2.3.4 C4. Learn embeddings

We train a skip-gram model with negative sampling on these walks and produce 2D embeddings ($d = 2$). The resulting coordinates are used to plot each graph with the original edges.

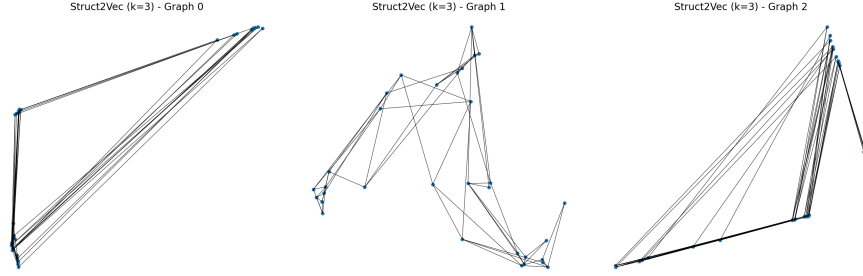


Figure 8: Struct2Vec embeddings for graphs 0, 1, and 2 (left to right), with $k = 3$ and $\text{top-}m = 10$.

2.4 Part D: Comparison questions (short answers)

1. For all three graphs, the original geometry (pos) preserves the geometric shape best because it is the ground-truth layout. Among the learned embeddings, the Node2Vec (BFS-biased) plots look most similar to the originals for graphs 1 and 2 since they keep nearby nodes along the curve close together, whereas Struct2Vec groups nodes by role and distorts the geometric shape.
2. Node2Vec mainly captures proximity/homophily: nodes that co-occur in short random walks or share local neighborhoods end up close in the embedding. Because walks from nearby nodes are similar, Node2Vec can overlap points from the same neighborhood, folding the shape (e.g., a circle can look like a semicircle); opposite points connected by a chord can get overlapped in the embedding.
3. Node2Vec struggles with structural similarity because its objective rewards co-occurrence in walks, so nodes that are structurally similar but far apart rarely appear together. As a result, same-role nodes in different regions are not pulled together.
4. Struct2Vec benefits nodes that share similar structural roles (e.g., hubs, bridges, peripheral/leaf nodes) even if they are far apart. If we squint at the plots, we can see nodes compacted into a handful of clusters (roughly 5–6 places), because the graph has only a few distinct node types based on degree and local neighborhood patterns. Struct2Vec groups those roles together, so many nodes collapse into a small number of locations in the embedding.

3 3. Graph-Based Sentiment Analysis with Node2Vec

3.1 Part A: Construct a word co-occurrence graph

3.1.1 A1. Text preprocessing

We use the IMDb sentiment dataset (aclImdb) with the provided train/test split, and to keep runtime reasonable we subsample 2000 positive and 2000 negative reviews for training and 1000 per class for testing. Each review is lowercased, punctuation/special characters are removed, text is tokenized on whitespace, and English stop words (sklearn list) are removed.

3.1.2 A2. Build the word graph

We build an undirected word co-occurrence graph from the training corpus with vocabulary size $V = 5000$ (most frequent tokens). An edge is added between words that occur within a context window of $w = 2$ in the same review, and we weight edges using positive PMI so frequent words are down-weighted while informative co-occurrences are emphasized. The motivation is that hubs like *movie*, *film*, or *actor* appear with almost everything (e.g., *awful movie*), which is not very informative; PMI reduces weights to such common nouns while giving higher weight to rarer, sentiment-bearing pairs such as *awful-boring*.

3.2 Part B: Learn word embeddings using Node2Vec

3.2.1 B1. Apply Node2Vec

We run Node2Vec on the word graph with embedding dimension $d = 64$, walk length = 10, number of walks per node = 20, and parameters $p = 1.0$, $q = 0.5$ (BFS-like bias). Skip-gram with negative sampling (5 negatives, window size 2) is trained for 6 epochs.

3.2.2 B2. Inspect embeddings

Figure 9 shows a t-SNE plot of the top-200 frequent words. Positive and negative sentiment words tend to form small local clusters, but there is still overlap, which is expected because the embeddings are driven by co-occurrence context rather than sentiment labels.

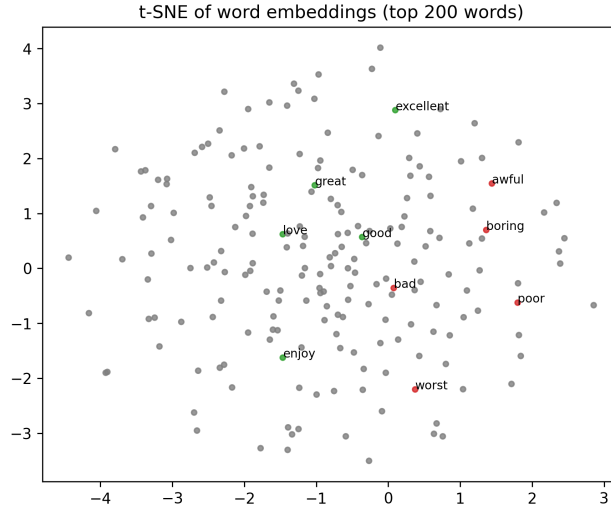


Figure 9: t-SNE of top-200 word embeddings; highlighted sentiment words are annotated.

3.3 Part C: Text representation using graph embeddings

3.3.1 C1. Document embeddings

Each document is represented by a simple average (mean pooling) of its word embeddings. This ignores word order and relative importance, but yields one 64-D vector per review; documents with no in-vocabulary tokens are mapped to the zero vector.

3.4 Part D: Sentiment classification

3.4.1 D1. Train a classifier

We train a logistic regression classifier on the mean-pooled document embeddings. On the test split, we obtain an accuracy of **80.95%**. The implementation is shown in Listing 13.

3.5 Part E: Short discussion

1. The word co-occurrence graph captures contextual relationships between words (who appears near whom), which bag-of-words ignores. It can encode local syntactic/semantic structure and neighborhood similarity beyond raw frequency.
2. The Node2Vec parameters p and q control the random-walk bias: smaller q favors BFS-like walks (local neighborhoods) while larger q encourages

DFS-like exploration. This changes whether embeddings emphasize local context similarity or broader structural proximity.

3. Compared to TF-IDF, this graph-based approach can capture richer contextual similarity but is more computationally expensive and sensitive to hyperparameters and graph construction choices. It also requires extra steps (walks and training) and may underperform on small datasets if the graph is noisy.

A Code

Listing 1: Part A code

```
1 from torch_geometric.datasets import KarateClub
2
3 dataset = KarateClub()
4 data = dataset[0]
5 print(f"Number of nodes: {data.num_nodes}")
6 print(f"Number of edges: {data.num_edges}")
7 if data.x is not None:
8     print(f"data.x shape: {data.x.shape}")
9 else:
10    print("data.x shape: None")
11 print(f"data.edge_index shape: {data.edge_index.shape}")
12 print(f"Number of node labels/classes: {dataset.num_classes}")
    "
```

B Graph Visualization Code

Listing 2: Part A graph visualization

```
1 import matplotlib
2 matplotlib.use("Agg")
3 import matplotlib.pyplot as plt
4 import networkx as nx
5 from torch_geometric.datasets import KarateClub
6 from torch_geometric.utils import to_networkx
7
8 dataset = KarateClub()
9 data = dataset[0]
10
11 # Convert PyG graph to NetworkX
12 G = to_networkx(data, to_undirected=True)
13
14 # Node colors based on labels
15 labels = data.y.tolist()
```

```

16
17 # Define colors to match the figure legend
18 color_map = {
19     0: "#66C2A5",
20     1: "#FC8D62",
21     2: "#8DA0CB",
22     3: "#FFD92F",
23 }
24 node_colors = [color_map[label] for label in labels]
25
26 # Draw the graph
27 pos = nx.spring_layout(G, seed=42)
28 plt.figure(figsize=(6, 5))
29 nx.draw(
30     G,
31     pos,
32     node_color=node_colors,
33     with_labels=True,
34     node_size=400,
35     font_size=8,
36 )
37 plt.title("Karate Club Graph")
38 plt.tight_layout()
39 plt.savefig("karate_club_graph.png", dpi=300)

```

C GeometricShapes Visualization Code

Listing 3: GeometricShapes visualization (Part 2A)

```

1 import matplotlib
2 matplotlib.use("Agg")
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from torch_geometric.datasets import GeometricShapes
6 from torch_geometric.transforms import FaceToEdge
7
8
9 def plot_graph(data, idx):
10     pos = data.pos.cpu().numpy()
11     edge_index = data.edge_index.cpu().numpy()
12     dim = pos.shape[1]
13
14     fig = plt.figure(figsize=(5, 5))
15     if dim == 3:
16         ax = fig.add_subplot(111, projection="3d")
17         ax.scatter(pos[:, 0], pos[:, 1], pos[:, 2], s=8, c="
            tab:blue")
18         for u, v in edge_index.T:

```

```

19         xs = [pos[u, 0], pos[v, 0]]
20         ys = [pos[u, 1], pos[v, 1]]
21         zs = [pos[u, 2], pos[v, 2]]
22         ax.plot(xs, ys, zs, color="black", linewidth
23                 =0.5, alpha=0.6)
24     else:
25         ax = fig.add_subplot(111)
26         ax.scatter(pos[:, 0], pos[:, 1], s=8, c="tab:blue")
27         for u, v in edge_index.T:
28             xs = [pos[u, 0], pos[v, 0]]
29             ys = [pos[u, 1], pos[v, 1]]
30             ax.plot(xs, ys, color="black", linewidth=0.5,
31                     alpha=0.6)
32         ax.set_aspect("equal", adjustable="box")
33
34         ax.set_title(f"GeometricShapes graph {idx}")
35         ax.axis("off")
36         return fig
37
38 # Load dataset with FaceToEdge so edge_index is populated
39 transform = FaceToEdge()
40 dataset = GeometricShapes(root="data/GeometricShapes",
41                             pre_transform=transform)
42
43 # Pick any 3 graphs
44 indices = [0, 1, 2]
45 for idx in indices:
46     fig = plot_graph(dataset[idx], idx)
47     fig.tight_layout()
48     fig.savefig(f"geometric_shapes_graph{idx}.png", dpi=200)
49     plt.close(fig)

```

D Node2Vec (Biased Walks + Skip-gram) Code

Listing 4: Node2Vec on GeometricShapes (Part 2B)

```

1 import math
2 import numpy as np
3 import torch
4 import torch.nn.functional as F
5 from torch import nn, optim
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 from torch_geometric.datasets import GeometricShapes
10 from torch_geometric.transforms import FaceToEdge

```

```

11
12
13 def build_undirected_adj(num_nodes, edge_index):
14     adj = [set() for _ in range(num_nodes)]
15     src, dst = edge_index
16     for u, v in zip(src, dst):
17         u = int(u)
18         v = int(v)
19         adj[u].add(v)
20         adj[v].add(u)
21     return adj
22
23
24 def node2vec_walk(start, walk_length, adj, p, q, rng):
25     walk = [start]
26     while len(walk) < walk_length:
27         v = walk[-1]
28         neighbors = list(adj[v])
29         if not neighbors:
30             break
31         if len(walk) == 1:
32             next_node = rng.choice(neighbors)
33         else:
34             t = walk[-2]
35             weights = []
36             t_neighbors = adj[t]
37             for x in neighbors:
38                 if x == t:
39                     w = 1.0 / p
40                 elif x in t_neighbors:
41                     w = 1.0
42                 else:
43                     w = 1.0 / q
44             weights.append(w)
45             weights = np.asarray(weights, dtype=np.float64)
46             probs = weights / weights.sum()
47             next_node = rng.choice(neighbors, p=probs)
48         walk.append(int(next_node))
49     return walk
50
51
52 def generate_walks(adj, num_walks_per_node, walk_length, p,
53 q, seed=42):
54     rng = np.random.default_rng(seed)
55     walks = []
56     nodes = list(range(len(adj)))
57     for _ in range(num_walks_per_node):
58         rng.shuffle(nodes)
59         for start in nodes:

```

```

59         walks.append(node2vec_walk(start, walk_length,
60                                   adj, p, q, rng))
61     return walks
62
63 def build_pairs(walks, window_size):
64     pairs = []
65     for walk in walks:
66         for i, center in enumerate(walk):
67             left = max(0, i - window_size)
68             right = min(len(walk), i + window_size + 1)
69             for j in range(left, right):
70                 if i == j:
71                     continue
72                 pairs.append((center, walk[j]))
73     return pairs
74
75
76 def build_negative_sampling_dist(walks, num_nodes, power
77                                 =0.75):
78     counts = np.zeros(num_nodes, dtype=np.float64)
79     for walk in walks:
80         for node in walk:
81             counts[node] += 1
82     counts = np.power(counts, power)
83     counts[counts == 0] = 1.0
84     probs = counts / counts.sum()
85     return torch.tensor(probs, dtype=torch.float32)
86
87 def train_skipgram(
88     num_nodes,
89     pairs,
90     neg_dist,
91     embedding_dim=2,
92     num_negatives=5,
93     epochs=50,
94     batch_size=256,
95     lr=0.01,
96     seed=42,
97 ):
98     torch.manual_seed(seed)
99     device = torch.device("cpu")
100
101     emb_in = nn.Embedding(num_nodes, embedding_dim).to(
102         device)
103     emb_out = nn.Embedding(num_nodes, embedding_dim).to(
104         device)
105     optimizer = optim.Adam(list(emb_in.parameters()) + list(
106         emb_out.parameters()), lr=lr)

```



```

104 pairs = np.asarray(pairs, dtype=np.int64)
105 num_pairs = len(pairs)
106
107
108 for epoch in range(epochs):
109     perm = np.random.permutation(num_pairs)
110     for start in range(0, num_pairs, batch_size):
111         idx = perm[start : start + batch_size]
112         batch = pairs[idx]
113         centers = torch.tensor(batch[:, 0], dtype=torch.
114                                long, device=device)
115         contexts = torch.tensor(batch[:, 1], dtype=torch.
116                                long, device=device)
117
118         center_vecs = emb_in(centers)
119         context_vecs = emb_out(contexts)
120         pos_scores = torch.sum(center_vecs *
121                                context_vecs, dim=1)
122         pos_loss = -F.logsigmoid(pos_scores).mean()
123
124         neg_samples = torch.multinomial(
125             neg_dist, centers.size(0) * num_negatives,
126             replacement=True
127         ).to(device)
128         neg_samples = neg_samples.view(centers.size(0),
129                                       num_negatives)
130         neg_vecs = emb_out(neg_samples)
131         neg_scores = torch.sum(center_vecs.unsqueeze(1)
132                                * neg_vecs, dim=2)
133         neg_loss = -F.logsigmoid(-neg_scores).mean()
134
135         loss = pos_loss + neg_loss
136         optimizer.zero_grad()
137         loss.backward()
138         optimizer.step()
139
140     return emb_in.weight.detach().cpu().numpy()
141
142
143 def plot_embedding(data, emb, title, out_path):
144     edge_index = data.edge_index.cpu().numpy()
145     fig, ax = plt.subplots(figsize=(5, 5))
146     for u, v in edge_index.T:
147         xs = [emb[u, 0], emb[v, 0]]
148         ys = [emb[u, 1], emb[v, 1]]
149         ax.plot(xs, ys, color="black", linewidth=0.5, alpha
150               =0.6)
151     ax.scatter(emb[:, 0], emb[:, 1], s=12, c="tab:blue")
152     ax.set_title(title)
153     ax.axis("off")

```

```

147     fig.tight_layout()
148     fig.savefig(out_path, dpi=200)
149     plt.close(fig)
150
151
152 if __name__ == "__main__":
153     # Node2Vec parameters (reported in the writeup)
154     p = 10
155     q_bfs = 0.5
156     q_dfs = 5
157     print(f"Node2Vec parameters: p={p}, q_bfs={q_bfs}, q_dfs={q_dfs}")
158
159     dataset = GeometricShapes(root="data/GeometricShapes",
160                               pre_transform=FaceToEdge())
161     indices = [0, 1, 2]
162
163     for idx in indices:
164         data = dataset[idx]
165         adj = build_undirected_adj(data.num_nodes, data.
166                                   edge_index)
167         for label, q in [("bfs", q_bfs), ("dfs", q_dfs)]:
168             walks = generate_walks(
169                 adj,
170                 num_walks_per_node=10,
171                 walk_length=20,
172                 p=p,
173                 q=q,
174                 seed=42,
175             )
176             pairs = build_pairs(walks, window_size=5)
177             neg_dist = build_negative_sampling_dist(walks,
178                                                     data.num_nodes)
179             emb = train_skipgram(
180                 data.num_nodes,
181                 pairs,
182                 neg_dist,
183                 embedding_dim=2,
184                 num_negatives=5,
185                 epochs=50,
186                 batch_size=256,
187                 lr=0.01,
188                 seed=42,
189             )
190             out_path = f"node2vec_graph{idx}_{label}.png"
191             title = f"Node2Vec ({label}) - Graph {idx}"
192             plot_embedding(data, emb, title, out_path)
193             print(
194                 f"Graph {idx} [{label}]: walks={len(walks)},
195                 pairs={len(pairs)}, saved {out_path}"
196             )

```

E Struct2Vec Code

Listing 5: Struct2Vec on GeometricShapes (Part 2C)

```

1 import math
2 import numpy as np
3 import torch
4 import torch.nn.functional as F
5 from torch import nn, optim
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 from torch_geometric.datasets import GeometricShapes
10 from torch_geometric.transforms import FaceToEdge
11
12
13 def build_undirected_adj(num_nodes, edge_index):
14     adj = [set() for _ in range(num_nodes)]
15     src, dst = edge_index
16     for u, v in zip(src, dst):
17         u = int(u)
18         v = int(v)
19         adj[u].add(v)
20         adj[v].add(u)
21     return adj
22
23
24 def k_hop_signatures(adj, k):
25     num_nodes = len(adj)
26     degrees = [len(adj[u]) for u in range(num_nodes)]
27     signatures = []
28     for u in range(num_nodes):
29         visited = {u}
30         frontier = [u]
31         for _ in range(k):
32             next_frontier = []
33             for v in frontier:
34                 for w in adj[v]:
35                     if w not in visited:
36                         visited.add(w)
37                         next_frontier.append(w)
38             frontier = next_frontier
39             if not frontier:
40                 break
41         sig = sorted(degrees[w] for w in visited)
42         signatures.append(sig)

```

```

43     return signatures
44
45
46 def gk_similarity(sig_u, sig_v):
47     len_u = len(sig_u)
48     len_v = len(sig_v)
49     L = max(len_u, len_v)
50     if L == 0:
51         return 0.0
52     if len_u < L:
53         sig_u = sig_u + [0] * (L - len_u)
54     if len_v < L:
55         sig_v = sig_v + [0] * (L - len_v)
56     diff = np.abs(np.array(sig_u, dtype=np.float64) - np.
57                   array(sig_v, dtype=np.float64)).sum()
58     return math.exp(-diff / L)
59
60 def build_struct2vec_graph(signatures, top_m=10):
61     n = len(signatures)
62     weighted_adj = [[] for _ in range(n)]
63     for u in range(n):
64         sims = []
65         for v in range(n):
66             if u == v:
67                 continue
68             s = gk_similarity(signatures[u], signatures[v])
69             sims.append((v, s))
70             sims.sort(key=lambda x: x[1], reverse=True)
71             if top_m is not None:
72                 sims = sims[:top_m]
73             weighted_adj[u] = sims
74     return weighted_adj
75
76
77 def weighted_walk(start, walk_length, weighted_adj, rng):
78     walk = [start]
79     while len(walk) < walk_length:
80         v = walk[-1]
81         neighbors = weighted_adj[v]
82         if not neighbors:
83             break
84         nodes = [n for n, _ in neighbors]
85         weights = np.array([w for _, w in neighbors], dtype=
86                             np.float64)
87         weights_sum = weights.sum()
88         if weights_sum == 0:
89             break
90         probs = weights / weights_sum
91         next_node = rng.choice(nodes, p=probs)

```

```

91         walk.append(int(next_node))
92     return walk
93
94
95 def generate_walks(weighted_adj, num_walks_per_node,
96 walk_length, seed=42):
97     rng = np.random.default_rng(seed)
98     walks = []
99     nodes = list(range(len(weighted_adj)))
100     for _ in range(num_walks_per_node):
101         rng.shuffle(nodes)
102         for start in nodes:
103             walks.append(weighted_walk(start, walk_length,
104                                     weighted_adj, rng))
105     return walks
106
107 def build_pairs(walks, window_size):
108     pairs = []
109     for walk in walks:
110         for i, center in enumerate(walk):
111             left = max(0, i - window_size)
112             right = min(len(walk), i + window_size + 1)
113             for j in range(left, right):
114                 if i == j:
115                     continue
116                 pairs.append((center, walk[j]))
117     return pairs
118
119 def build_negative_sampling_dist(walks, num_nodes, power
120 =0.75):
121     counts = np.zeros(num_nodes, dtype=np.float64)
122     for walk in walks:
123         for node in walk:
124             counts[node] += 1
125     counts = np.power(counts, power)
126     counts[counts == 0] = 1.0
127     probs = counts / counts.sum()
128     return torch.tensor(probs, dtype=torch.float32)
129
130 def train_skipgram(
131     num_nodes,
132     pairs,
133     neg_dist,
134     embedding_dim=2,
135     num_negatives=5,
136     epochs=50,
137     batch_size=256,

```

```

138     lr=0.01,
139     seed=42,
140 ):
141     torch.manual_seed(seed)
142     emb_in = nn.Embedding(num_nodes, embedding_dim)
143     emb_out = nn.Embedding(num_nodes, embedding_dim)
144     optimizer = optim.Adam(list(emb_in.parameters()) + list(
145         emb_out.parameters()), lr=lr)
146
147     pairs = np.asarray(pairs, dtype=np.int64)
148     num_pairs = len(pairs)
149
150     for epoch in range(epochs):
151         perm = np.random.permutation(num_pairs)
152         for start in range(0, num_pairs, batch_size):
153             idx = perm[start : start + batch_size]
154             batch = pairs[idx]
155             centers = torch.tensor(batch[:, 0], dtype=torch.
156                 long)
157             contexts = torch.tensor(batch[:, 1], dtype=torch.
158                 long)
159
160             center_vecs = emb_in(centers)
161             context_vecs = emb_out(contexts)
162             pos_scores = torch.sum(center_vecs *
163                 context_vecs, dim=1)
164             pos_loss = -F.logsigmoid(pos_scores).mean()
165
166             neg_samples = torch.multinomial(
167                 neg_dist, centers.size(0) * num_negatives,
168                 replacement=True
169             )
170             neg_samples = neg_samples.view(centers.size(0),
171                 num_negatives)
172             neg_vecs = emb_out(neg_samples)
173             neg_scores = torch.sum(center_vecs.unsqueeze(1)
174                 * neg_vecs, dim=2)
175             neg_loss = -F.logsigmoid(-neg_scores).mean()
176
177             loss = pos_loss + neg_loss
178             optimizer.zero_grad()
179             loss.backward()
180             optimizer.step()
181
182     return emb_in.weight.detach().cpu().numpy()
183
184 def plot_embedding(data, emb, title, out_path):
185     edge_index = data.edge_index.cpu().numpy()
186     fig, ax = plt.subplots(figsize=(5, 5))

```

```

181     for u, v in edge_index.T:
182         xs = [emb[u, 0], emb[v, 0]]
183         ys = [emb[u, 1], emb[v, 1]]
184         ax.plot(xs, ys, color="black", linewidth=0.5, alpha
                =0.6)
185     ax.scatter(emb[:, 0], emb[:, 1], s=12, c="tab:blue")
186     ax.set_title(title)
187     ax.axis("off")
188     fig.tight_layout()
189     fig.savefig(out_path, dpi=200)
190     plt.close(fig)
191
192
193 if __name__ == "__main__":
194     k = 3
195     top_m = 10
196     num_walks_per_node = 10
197     walk_length = 20
198     window_size = 5
199
200     dataset = GeometricShapes(root="data/GeometricShapes",
                                pre_transform=FaceToEdge())
201     indices = [0, 1, 2]
202
203     for idx in indices:
204         data = dataset[idx]
205         adj = build_undirected_adj(data.num_nodes, data.
                                    edge_index)
206         signatures = k_hop_signatures(adj, k=k)
207         weighted_adj = build_struct2vec_graph(signatures,
                                                top_m=top_m)
208
209         walks = generate_walks(weighted_adj,
                                num_walks_per_node, walk_length, seed=42)
210         pairs = build_pairs(walks, window_size=window_size)
211         neg_dist = build_negative_sampling_dist(walks, data.
                                                num_nodes)
212
213         emb = train_skipgram(
214             data.num_nodes,
215             pairs,
216             neg_dist,
217             embedding_dim=2,
218             num_negatives=5,
219             epochs=50,
220             batch_size=256,
221             lr=0.01,
222             seed=42,
223         )
224

```

```

225     out_path = f"struct2vec_graph{idx}.png"
226     title = f"Struct2Vec (k={k}) - Graph {idx}"
227     plot_embedding(data, emb, title, out_path)
228     print(
229         f"Graph {idx}: walks={len(walks)}, pairs={len(
230             pairs)}, saved {out_path}"
231     )

```

F Structural Features Code

Listing 6: Part C1 structural features

```

1  import math
2  from torch_geometric.datasets import KarateClub
3
4
5  def build_undirected_adj(num_nodes, edge_index):
6      adj = [set() for _ in range(num_nodes)]
7      for u, v in zip(edge_index[0], edge_index[1]):
8          u = int(u)
9          v = int(v)
10         if u == v:
11             continue
12         adj[u].add(v)
13         adj[v].add(u)
14     return adj
15
16
17  def bfs_distances(adj, source):
18      n = len(adj)
19      dist = [-1] * n
20      dist[source] = 0
21      queue = [source]
22      head = 0
23      while head < len(queue):
24          u = queue[head]
25          head += 1
26          for v in adj[u]:
27              if dist[v] == -1:
28                  dist[v] = dist[u] + 1
29                  queue.append(v)
30     return dist
31
32
33  def betweenness centrality(adj):
34      n = len(adj)
35      bc = [0.0] * n
36      for s in range(n):

```



```

37     stack = []
38     pred = [[] for _ in range(n)]
39     sigma = [0.0] * n
40     dist = [-1] * n
41     sigma[s] = 1.0
42     dist[s] = 0
43     queue = [s]
44     head = 0
45     while head < len(queue):
46         v = queue[head]
47         head += 1
48         stack.append(v)
49         for w in adj[v]:
50             if dist[w] < 0:
51                 dist[w] = dist[v] + 1
52                 queue.append(w)
53             if dist[w] == dist[v] + 1:
54                 sigma[w] += sigma[v]
55                 pred[w].append(v)
56     delta = [0.0] * n
57     while stack:
58         w = stack.pop()
59         for v in pred[w]:
60             if sigma[w] != 0:
61                 delta[v] += (sigma[v] / sigma[w]) * (1.0
62                     + delta[w])
63             if w != s:
64                 bc[w] += delta[w]
65     for i in range(n):
66         bc[i] /= 2.0
67     if n > 2:
68         scale = 1.0 / ((n - 1) * (n - 2) / 2.0)
69         bc = [x * scale for x in bc]
70     return bc
71
72 def closeness centrality(adj):
73     n = len(adj)
74     cc = [0.0] * n
75     for u in range(n):
76         dist = bfs_distances(adj, u)
77         total = 0
78         reachable = 0
79         for d in dist:
80             if d > 0:
81                 total += d
82                 reachable += 1
83         if total > 0 and reachable > 0:
84             cc[u] = reachable / total
85     return cc

```

```

86
87
88 def pagerank(adj, damping=0.85, max_iter=100, tol=1e-6):
89     n = len(adj)
90     pr = [1.0 / n] * n
91     out_deg = [len(adj[u]) for u in range(n)]
92     for _ in range(max_iter):
93         new_pr = [(1.0 - damping) / n] * n
94         for u in range(n):
95             if out_deg[u] == 0:
96                 share = damping * pr[u] / n
97                 for v in range(n):
98                     new_pr[v] += share
99             else:
100                 share = damping * pr[u] / out_deg[u]
101                 for v in adj[u]:
102                     new_pr[v] += share
103         diff = 0.0
104         for i in range(n):
105             diff += abs(new_pr[i] - pr[i])
106         pr = new_pr
107         if diff < tol:
108             break
109     return pr
110
111
112 def eigenvector_centrality(adj, max_iter=100, tol=1e-6):
113     n = len(adj)
114     x = [1.0] * n
115     for _ in range(max_iter):
116         x_new = [0.0] * n
117         for u in range(n):
118             for v in adj[u]:
119                 x_new[u] += x[v]
120         norm = math.sqrt(sum(val * val for val in x_new))
121         if norm == 0:
122             break
123         x_new = [val / norm for val in x_new]
124         diff = sum(abs(x_new[i] - x[i]) for i in range(n))
125         x = x_new
126         if diff < tol:
127             break
128     return x
129
130
131 def clustering_coefficient(adj):
132     n = len(adj)
133     cc = [0.0] * n
134     for u in range(n):
135         neighbors = list(adj[u])

```

```

136         k = len(neighbors)
137         if k < 2:
138             cc[u] = 0.0
139             continue
140         links = 0
141         for i in range(k):
142             for j in range(i + 1, k):
143                 if neighbors[j] in adj[neighbors[i]]:
144                     links += 1
145             cc[u] = (2.0 * links) / (k * (k - 1))
146     return cc
147
148
149 def zscore_columns(matrix):
150     n = len(matrix)
151     m = len(matrix[0])
152     means = [0.0] * m
153     stds = [0.0] * m
154     for j in range(m):
155         s = 0.0
156         for i in range(n):
157             s += matrix[i][j]
158         means[j] = s / n
159     for j in range(m):
160         s = 0.0
161         for i in range(n):
162             diff = matrix[i][j] - means[j]
163             s += diff * diff
164         stds[j] = math.sqrt(s / n)
165         if stds[j] == 0:
166             stds[j] = 1.0
167     for i in range(n):
168         for j in range(m):
169             matrix[i][j] = (matrix[i][j] - means[j]) / stds[
170                 j]
171     return matrix
172
173 def compute_structural_features(adj, node_order=None,
174                                normalize=True):
175     """
176     Compute structural features for each node and stack them
177     into a matrix.
178     Features: degree, betweenness, closeness, PageRank,
179     eigenvector, clustering.
180     """
181     n = len(adj)
182     if node_order is None:
183         node_order = list(range(n))

```

```

182     degree = [len(adj[u]) for u in range(n)]
183     betweenness = betweenness centrality(adj)
184     closeness = closeness centrality(adj)
185     pr = pagerank(adj)
186     eigen = eigenvector centrality(adj)
187     cluster = clustering_coefficient(adj)
188
189     X = []
190     for node in node_order:
191         X.append(
192             [
193                 degree[node],
194                 betweenness[node],
195                 closeness[node],
196                 pr[node],
197                 eigen[node],
198                 cluster[node],
199             ]
200         )
201
202     if normalize:
203         X = zscore_columns(X)
204
205     return X
206
207
208 def main():
209     dataset = KarateClub()
210     data = dataset[0]
211
212     adj = build_undirected_adj(data.num_nodes, data.
213                               edge_index)
214     node_order = list(range(data.num_nodes))
215     X_struct = compute_structural_features(adj, node_order=
216                                           node_order, normalize=True)
217
218     print("Structural embedding matrix shape:", (len(
219           X_struct), len(X_struct[0])))
220     print("First 5 rows:")
221     for row in X_struct[:5]:
222         print(row)
223
224 if __name__ == "__main__":
225     main()

```

G Structural Embeddings Visualization Code

Listing 7: Part C2 PCA visualization

```

1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 import matplotlib.colors as mcolors
6 from torch_geometric.datasets import KarateClub
7
8 from structural_features import build_undirected_adj,
   compute_structural_features
9
10
11 def pca_2d(features):
12     X = np.array(features, dtype=float)
13     X_centered = X - X.mean(axis=0, keepdims=True)
14     cov = np.cov(X_centered, rowvar=False)
15     eigvals, eigvecs = np.linalg.eigh(cov)
16     order = np.argsort(eigvals)[::-1]
17     eigvals_sorted = eigvals[order]
18     eigvecs_sorted = eigvecs[:, order]
19     components = eigvecs_sorted[:, :2]
20     emb_2d = X_centered @ components
21     return emb_2d, components, eigvals_sorted
22
23
24 def main():
25     dataset = KarateClub()
26     data = dataset[0]
27
28     adj = build_undirected_adj(data.num_nodes, data.
   edge_index)
29     node_order = list(range(data.num_nodes))
30
31     X_struct = compute_structural_features(
32         adj,
33         node_order=node_order,
34         should_normalize=True,
35     )
36     emb_2d, components, eigvals_sorted = pca_2d(X_struct)
37
38     feature_names = [
39         "degree",
40         "betweenness",
41         "closeness",
42         "pagerank",
43         "eigenvector",
44         "clustering",
45     ]
46

```

```

47     if components.shape[0] != len(feature_names):
48         print(
49             f"[WARN] components has {components.shape[0]}
50             rows (features), but feature_names has {len(
51                 feature_names)}. "
52             "Update feature_names to match
53             compute_structural_features column order/
54             length."
55         )
56
57     total_var = float(np.sum(eigvals_sorted)) if np.sum(
58         eigvals_sorted) != 0 else 1.0
59     evr = eigvals_sorted / total_var
60     print("\nExplained variance ratio:")
61     print(f"    PC1: {evr[0]:.4f}")
62     print(f"    PC2: {evr[1]:.4f}")
63
64     for pc_idx in range(2):
65         pc_name = f"PC{pc_idx + 1}"
66         print(f"\n{pc_name} loadings (feature weights):")
67
68         pairs = []
69         for i in range(min(len(feature_names), components.
70             shape[0])):
71             pairs.append((feature_names[i], float(components
72                 [i, pc_idx])))
73
74         pairs.sort(key=lambda x: abs(x[1]), reverse=True)
75
76         for fname, w in pairs:
77             print(f"    {fname:12s} {w:+.6f}")
78
79     labels = data.y.tolist()
80     classes = sorted(set(int(v) for v in labels))
81     class_to_idx = {c: i for i, c in enumerate(classes)}
82     labels_idx = np.array([class_to_idx[int(v)] for v in
83         labels], dtype=int)
84
85     cmap = plt.cm.get_cmap("Set2", len(classes))
86     bounds = np.arange(-0.5, len(classes) + 0.5, 1)
87     norm = mcolors.BoundaryNorm(bounds, cmap.N)
88
89     plt.figure(figsize=(6, 5))
90     scatter = plt.scatter(
91         emb_2d[:, 0],
92         emb_2d[:, 1],
93         c=labels_idx,
94         cmap=cmap,
95         norm=norm,
96         s=70,

```

```

89         edgecolors="k",
90         linewidths=0.4,
91     )
92     plt.title("PCA of Structural Features")
93     plt.xlabel("PC1")
94     plt.ylabel("PC2")
95
96     cbar = plt.colorbar(scatter, ticks=np.arange(len(classes)
97     ))
98     cbar.set_ticklabels([str(c) for c in classes])
99     cbar.set_label("data.y class")
100
101     plt.tight_layout()
102     plt.savefig("structural_pca.png", dpi=200)
103     plt.close()
104
105 if __name__ == "__main__":
106     main()

```

H Node Classification Code

Listing 8: Part C3 node classification using structural features

```

1  import random
2  import numpy as np
3  import torch
4  from torch import nn
5  from torch_geometric.datasets import KarateClub
6  from sklearn.model_selection import StratifiedKFold
7
8  from structural_features import build_undirected_adj,
9     compute_structural_features
10
11 def set_seed(seed: int) -> None:
12     random.seed(seed)
13     np.random.seed(seed)
14     torch.manual_seed(seed)
15
16
17 def build_mlp(input_dim, num_classes):
18     return nn.Sequential(
19         nn.Linear(input_dim, 16),
20         nn.ReLU(),
21         nn.Linear(16, num_classes),
22     )
23

```

```

24
25 def build_logreg(input_dim, num_classes):
26     return nn.Linear(input_dim, num_classes)
27
28
29 def train_classifier(model, x, y, train_idx, epochs=500, lr
=0.05, weight_decay=1e-4):
30     optimizer = torch.optim.Adam(model.parameters(), lr=lr,
        weight_decay=weight_decay)
31     criterion = nn.CrossEntropyLoss()
32
33     for _ in range(epochs):
34         model.train()
35         logits = model(x)
36         loss = criterion(logits[train_idx], y[train_idx])
37         optimizer.zero_grad()
38         loss.backward()
39         optimizer.step()
40
41     return model
42
43
44 def accuracy(model, x, y, idx):
45     model.eval()
46     with torch.no_grad():
47         preds = model(x).argmax(dim=1)
48         correct = (preds[idx] == y[idx]).sum().item()
49         total = idx.numel()
50     return correct / total if total > 0 else 0.0
51
52
53 def main():
54     set_seed(42)
55
56     dataset = KarateClub()
57     data = dataset[0]
58
59     adj = build_undirected_adj(data.num_nodes, data.
        edge_index)
60     node_order = list(range(data.num_nodes))
61     x_struct = compute_structural_features(
62         adj,
63         node_order=node_order,
64         should_normalize=True,
65     )
66
67     x = torch.tensor(x_struct, dtype=torch.float32)
68     y = data.y.clone().long()
69
70     num_classes = dataset.num_classes

```



```

71     skf = StratifiedKFold(n_splits=5, shuffle=True,
72                           random_state=42)
73
74     train_accuracies = []
75     test_accuracies = []
76
77     def eval_model(build_fn, label):
78         train_accuracies.clear()
79         test_accuracies.clear()
80
81         for fold, (train_idx_np, test_idx_np) in enumerate(
82             skf.split(x, y)):
83             set_seed(42 + fold)
84             train_idx = torch.tensor(train_idx_np, dtype=
85                                     torch.long)
86             test_idx = torch.tensor(test_idx_np, dtype=torch
87                                     .long)
88
89             model = build_fn(x.size(1), num_classes)
90             model = train_classifier(model, x, y, train_idx)
91
92             train_acc = accuracy(model, x, y, train_idx)
93             test_acc = accuracy(model, x, y, test_idx)
94
95             train_accuracies.append(train_acc)
96             test_accuracies.append(test_acc)
97
98             print(f"{label} Fold {fold + 1}: Train acc = {
99                 train_acc:.4f}, Test acc = {test_acc:.4f}")
100
101         print(f"\n{n{label} overall performance:")
102         print(f"Train accuracy: {np.mean(train_accuracies)
103               :.4f}      {np.std(train_accuracies):.4f}")
104         print(f"Test accuracy: {np.mean(test_accuracies):.4
105               f}      {np.std(test_accuracies):.4f}\\n")
106
107     eval_model(build_mlp, "MLP")
108     eval_model(build_logreg, "LogReg")
109
110 if __name__ == "__main__":
111     main()

```

I Node Features Classification Code

Listing 9: Part D1 node classification using node features

```

1 import random
2 import numpy as np
3 import torch
4 from torch import nn
5 from torch_geometric.datasets import KarateClub
6 from sklearn.model_selection import StratifiedKFold
7
8
9 def set_seed(seed: int) -> None:
10     random.seed(seed)
11     np.random.seed(seed)
12     torch.manual_seed(seed)
13
14
15 def build_mlp(input_dim, num_classes):
16     return nn.Sequential(
17         nn.Linear(input_dim, 16),
18         nn.ReLU(),
19         nn.Linear(16, num_classes),
20     )
21
22
23 def build_logreg(input_dim, num_classes):
24     return nn.Linear(input_dim, num_classes)
25
26
27 def train_classifier(model, x, y, train_idx, epochs=500, lr
=0.05, weight_decay=1e-4):
28     optimizer = torch.optim.Adam(model.parameters(), lr=lr,
        weight_decay=weight_decay)
29     criterion = nn.CrossEntropyLoss()
30
31     for _ in range(epochs):
32         model.train()
33         logits = model(x)
34         loss = criterion(logits[train_idx], y[train_idx])
35         optimizer.zero_grad()
36         loss.backward()
37         optimizer.step()
38
39     return model
40
41
42 def accuracy(model, x, y, idx):
43     model.eval()
44     with torch.no_grad():
45         preds = model(x).argmax(dim=1)
46         correct = (preds[idx] == y[idx]).sum().item()
47         total = idx.numel()

```

```

48     return correct / total if total > 0 else 0.0
49
50
51 def main():
52     set_seed(42)
53
54     dataset = KarateClub()
55     data = dataset[0]
56
57     x = data.x.clone().float()
58     y = data.y.clone().long()
59
60     num_classes = dataset.num_classes
61
62     skf = StratifiedKFold(n_splits=5, shuffle=True,
63                           random_state=42)
64     train_accuracies = []
65     test_accuracies = []
66
67     def eval_model(build_fn, label):
68         train_accuracies.clear()
69         test_accuracies.clear()
70
71         for fold, (train_idx_np, test_idx_np) in enumerate(
72             skf.split(x, y)):
73             set_seed(42 + fold)
74             train_idx = torch.tensor(train_idx_np, dtype=
75                                     torch.long)
76             test_idx = torch.tensor(test_idx_np, dtype=torch
77                                     .long)
78
79             model = build_fn(x.size(1), num_classes)
80             model = train_classifier(model, x, y, train_idx)
81
82             train_acc = accuracy(model, x, y, train_idx)
83             test_acc = accuracy(model, x, y, test_idx)
84
85             train_accuracies.append(train_acc)
86             test_accuracies.append(test_acc)
87
88             print(f"{label} Fold {fold + 1}: Train acc = {
89                   train_acc:.4f}, Test acc = {test_acc:.4f}")
90
91     print(f"\n{label} overall performance:")
92     print(f"Train accuracy: {np.mean(train_accuracies)
93           :.4f}      {np.std(train_accuracies):.4f}")
94     print(f"Test accuracy: {np.mean(test_accuracies):.4
95           f}      {np.std(test_accuracies):.4f}\n")
96
97     eval_model(build_mlp, "MLP")

```

```

91     eval_model(build_logreg, "LogReg")
92
93
94 if __name__ == "__main__":
95     main()

```

J Node Features PCA Code

Listing 10: Part D2 PCA visualization of node features

```

1  import matplotlib
2  matplotlib.use("Agg")
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from torch_geometric.datasets import KarateClub
6
7
8  def pca_2d(features):
9      x = np.array(features, dtype=float)
10     x_centered = x - x.mean(axis=0, keepdims=True)
11     cov = np.cov(x_centered, rowvar=False)
12     eigvals, eigvecs = np.linalg.eigh(cov)
13     order = np.argsort(eigvals)[::-1]
14     components = eigvecs[:, order][:, :2]
15     emb_2d = x_centered @ components
16     return emb_2d
17
18
19 def main():
20     dataset = KarateClub()
21     data = dataset[0]
22
23     x = data.x.cpu().numpy()
24     labels = data.y.cpu().numpy()
25
26     emb_2d = pca_2d(x)
27
28     plt.figure(figsize=(5, 4))
29     plt.scatter(emb_2d[:, 0], emb_2d[:, 1], c=labels, cmap=
        plt.cm.Set2, s=60, edgecolors="k")
30     plt.xlabel("PC1")
31     plt.ylabel("PC2")
32     plt.title("PCA of Node Features")
33     plt.tight_layout()
34     plt.savefig("node_features_pca.png", dpi=300)
35
36
37 if __name__ == "__main__":

```

```
38     main()
```

K Combined Features Classification Code

Listing 11: Part E node classification using combined features

```
1  import random
2  import numpy as np
3  import torch
4  from torch import nn
5  from torch_geometric.datasets import KarateClub
6  from sklearn.model_selection import StratifiedKFold
7
8  from structural_features import build_undirected_adj,
   compute_structural_features
9
10
11 def set_seed(seed: int) -> None:
12     random.seed(seed)
13     np.random.seed(seed)
14     torch.manual_seed(seed)
15
16
17 def build_mlp(input_dim, num_classes):
18     return nn.Sequential(
19         nn.Linear(input_dim, 16),
20         nn.ReLU(),
21         nn.Linear(16, num_classes),
22     )
23
24
25 def build_logreg(input_dim, num_classes):
26     return nn.Linear(input_dim, num_classes)
27
28
29 def train_classifier(model, x, y, train_idx, epochs=500, lr
   =0.05, weight_decay=1e-4):
30     optimizer = torch.optim.Adam(model.parameters(), lr=lr,
   weight_decay=weight_decay)
31     criterion = nn.CrossEntropyLoss()
32
33     for _ in range(epochs):
34         model.train()
35         logits = model(x)
36         loss = criterion(logits[train_idx], y[train_idx])
37         optimizer.zero_grad()
38         loss.backward()
39         optimizer.step()
```

```

40     return model
41
42
43
44 def accuracy(model, x, y, idx):
45     model.eval()
46     with torch.no_grad():
47         preds = model(x).argmax(dim=1)
48         correct = (preds[idx] == y[idx]).sum().item()
49         total = idx.numel()
50     return correct / total if total > 0 else 0.0
51
52
53 def main():
54     set_seed(42)
55
56     dataset = KarateClub()
57     data = dataset[0]
58
59     adj = build_undirected_adj(data.num_nodes, data.
60                               edge_index)
61     node_order = list(range(data.num_nodes))
62     x_struct = compute_structural_features(
63         adj,
64         node_order=node_order,
65         should_normalize=True,
66     )
67
68     x_struct = torch.tensor(x_struct, dtype=torch.float32)
69     x_attr = data.x.clone().float()
70
71     x = torch.cat([x_struct, x_attr], dim=1)
72     y = data.y.clone().long()
73
74     num_classes = dataset.num_classes
75
76     skf = StratifiedKFold(n_splits=5, shuffle=True,
77                           random_state=42)
78     train_accuracies = []
79     test_accuracies = []
80
81     def eval_model(build_fn, label):
82         train_accuracies.clear()
83         test_accuracies.clear()
84
85         for fold, (train_idx_np, test_idx_np) in enumerate(
86             skf.split(x, y)):
87             set_seed(42 + fold)
88             train_idx = torch.tensor(train_idx_np, dtype=
89                                     torch.long)

```

```

86         test_idx = torch.tensor(test_idx_np, dtype=torch
87                                   .long)
88
89         model = build_fn(x.size(1), num_classes)
90         model = train_classifier(model, x, y, train_idx)
91
92         train_acc = accuracy(model, x, y, train_idx)
93         test_acc = accuracy(model, x, y, test_idx)
94
95         train_accuracies.append(train_acc)
96         test_accuracies.append(test_acc)
97
98         print(f"{label} Fold {fold + 1}: Train acc = {
99               train_acc:.4f}, Test acc = {test_acc:.4f}")
100
101     print(f"\n{n{label} overall performance:")
102     print(f"Train accuracy: {np.mean(train_accuracies)
103           :.4f}      {np.std(train_accuracies):.4f}")
104     print(f"Test accuracy: {np.mean(test_accuracies):.4
105           f}      {np.std(test_accuracies):.4f}\\n")
106
107     eval_model(build_mlp, "MLP")
108     eval_model(build_logreg, "LogReg")
109
110 if __name__ == "__main__":
111     main()

```

L Cora Classification Code

Listing 12: Part F Cora node classification

```

1  import random
2  from pathlib import Path
3
4  import numpy as np
5  import torch
6  from torch import nn
7  from torch_geometric.datasets import Planetoid
8
9  from structural_features import build_undirected_adj,
10     compute_structural_features
11
12 def set_seed(seed: int) -> None:
13     random.seed(seed)
14     np.random.seed(seed)
15     torch.manual_seed(seed)

```

```

16
17
18 def build_mlp(input_dim: int, num_classes: int) -> nn.Module
19 :
20     return nn.Sequential(
21         nn.Linear(input_dim, 64),
22         nn.ReLU(),
23         nn.Dropout(0.5),
24         nn.Linear(64, num_classes),
25     )
26
27 def accuracy_from_logits(logits: torch.Tensor, y: torch.
28     Tensor, mask: torch.Tensor) -> float:
29     preds = logits.argmax(dim=1)
30     correct = (preds[mask] == y[mask]).sum().item()
31     total = int(mask.sum().item())
32     return correct / total if total > 0 else 0.0
33
34 def train_with_early_stopping(
35     model: nn.Module,
36     x: torch.Tensor,
37     y: torch.Tensor,
38     train_mask: torch.Tensor,
39     val_mask: torch.Tensor,
40     test_mask: torch.Tensor,
41     epochs: int = 500,
42     lr: float = 0.01,
43     weight_decay: float = 5e-4,
44     patience: int = 50,
45 ):
46     optimizer = torch.optim.Adam(model.parameters(), lr=lr,
47         weight_decay=weight_decay)
48     criterion = nn.CrossEntropyLoss()
49
50     best_val = -1.0
51     best_state = None
52     patience_left = patience
53
54     for _ in range(epochs):
55         model.train()
56         logits = model(x)
57         loss = criterion(logits[train_mask], y[train_mask])
58         optimizer.zero_grad()
59         loss.backward()
60         optimizer.step()
61
62         model.eval()
63         with torch.no_grad():

```



```

63         val_logits = model(x)
64         val_acc = accuracy_from_logits(val_logits, y,
                                         val_mask)
65
66         if val_acc > best_val + 1e-6:
67             best_val = val_acc
68             best_state = {k: v.detach().cpu().clone() for k,
                           v in model.state_dict().items()}
69             patience_left = patience
70         else:
71             patience_left -= 1
72             if patience_left == 0:
73                 break
74
75         if best_state is not None:
76             model.load_state_dict(best_state)
77
78     model.eval()
79     with torch.no_grad():
80         logits = model(x)
81         val_acc = accuracy_from_logits(logits, y, val_mask)
82         test_acc = accuracy_from_logits(logits, y, test_mask
                                         )
83
84     return val_acc, test_acc
85
86
87 def load_or_compute_structural_features(data, cache_path:
Path) -> torch.Tensor:
88     if cache_path.exists():
89         cached = np.load(cache_path)
90         if cached.shape[0] == data.num_nodes:
91             return torch.from_numpy(cached).float()
92
93     adj = build_undirected_adj(data.num_nodes, data.
edge_index)
94     node_order = list(range(data.num_nodes))
95     x_struct = compute_structural_features(
96         adj,
97         node_order=node_order,
98         should_normalize=True,
99     )
100     x_struct = np.asarray(x_struct, dtype=np.float32)
101
102     cache_path.parent.mkdir(parents=True, exist_ok=True)
103     np.save(cache_path, x_struct)
104
105     return torch.from_numpy(x_struct).float()
106
107

```

```

108 def main():
109     set_seed(42)
110
111     dataset = Planetoid(root="data/Planetoid", name="Cora")
112     data = dataset[0]
113
114     x_attr = data.x.clone().float()
115     y = data.y.clone().long()
116
117     cache_path = Path("data/cora_structural_features.npy")
118     x_struct = load_or_compute_structural_features(data,
119                                                    cache_path)
120     x_comb = torch.cat([x_struct, x_attr], dim=1)
121
122     train_mask = data.train_mask
123     val_mask = data.val_mask
124     test_mask = data.test_mask
125
126     settings = [
127         ("Structural only", x_struct),
128         ("Node features only", x_attr),
129         ("Combined", x_comb),
130     ]
131
132     for name, x in settings:
133         model = build_mlp(x.size(1), dataset.num_classes)
134         val_acc, test_acc = train_with_early_stopping(
135             model,
136             x,
137             y,
138             train_mask,
139             val_mask,
140             test_mask,
141         )
142         print(f"{name}: Val acc = {val_acc:.4f}, Test acc = {test_acc:.4f}")
143
144 if __name__ == "__main__":
145     main()

```

M Graph-Based Sentiment Analysis Code

Listing 13: Sentiment analysis with Node2Vec (Part 3)

```

1 import re
2 from collections import Counter, defaultdict
3 from pathlib import Path

```

```

4
5 import numpy as np
6 import torch
7 import torch.nn.functional as F
8 from torch import nn, optim
9 from sklearn.feature_extraction.text import
    ENGLISH_STOP_WORDS
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.manifold import TSNE
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15
16
17 def load_imdb_split(base_dir: Path, split: str,
    max_per_class: int | None = None):
18     texts, labels = [], []
19     for label, subdir in [(1, "pos"), (0, "neg")]:
20         dir_path = base_dir / split / subdir
21         files = sorted(dir_path.glob("*.txt"))
22         if max_per_class is not None:
23             files = files[:max_per_class]
24         for path in files:
25             texts.append(path.read_text(encoding="utf-8",
                errors="ignore"))
26             labels.append(label)
27     return texts, labels
28
29
30 def preprocess(text: str, stopwords: set[str]):
31     text = text.lower()
32     text = re.sub(r"[^a-z0-9\s]", " ", text)
33     tokens = [t for t in text.split() if t and t not in
        stopwords]
34     return tokens
35
36
37 def build_vocab(tokenized_texts, vocab_size: int):
38     counter = Counter()
39     for tokens in tokenized_texts:
40         counter.update(tokens)
41     most_common = counter.most_common(vocab_size)
42     vocab = {word: idx for idx, (word, _) in enumerate(
        most_common)}
43     freqs = np.array([count for _, count in most_common],
        dtype=np.float64)
44     return vocab, freqs
45
46

```

```

47 def build_cooc_graph(tokenized_texts, vocab, window_size:
    int):
48     edge_counts = defaultdict(int)
49     word_counts = np.zeros(len(vocab), dtype=np.int64)
50     total_tokens = 0
51     for tokens in tokenized_texts:
52         filtered = [t for t in tokens if t in vocab]
53         for token in filtered:
54             word_counts[vocab[token]] += 1
55         total_tokens += len(filtered)
56         for i, token in enumerate(filtered):
57             u = vocab[token]
58             for j in range(i + 1, min(i + window_size + 1,
                len(filtered))):
59                 v = vocab[filtered[j]]
60                 if u == v:
61                     continue
62                 a, b = (u, v) if u < v else (v, u)
63                 edge_counts[(a, b)] += 1
64
65     total_pairs = sum(edge_counts.values())
66     adj = [defaultdict(float) for _ in range(len(vocab))]
67     for (u, v), count in edge_counts.items():
68         p_uv = count / total_pairs
69         p_u = word_counts[u] / total_tokens
70         p_v = word_counts[v] / total_tokens
71         pmi = np.log(p_uv / (p_u * p_v))
72         if pmi <= 0.0:
73             continue
74         adj[u][v] = pmi
75         adj[v][u] = pmi
76     return adj
77
78
79 def node2vec_walk(start, walk_length, adj, p, q, rng):
80     walk = [start]
81     while len(walk) < walk_length:
82         v = walk[-1]
83         neighbors = list(adj[v].keys())
84         if not neighbors:
85             break
86         if len(walk) == 1:
87             weights = np.array([adj[v][x] for x in neighbors
                ], dtype=np.float64)
88         else:
89             t = walk[-2]
90             t_neighbors = adj[t]
91             weights = []
92             for x in neighbors:
93                 w = adj[v][x]

```

```

94         if x == t:
95             w *= 1.0 / p
96         elif x in t_neighbors:
97             w *= 1.0
98         else:
99             w *= 1.0 / q
100         weights.append(w)
101         weights = np.asarray(weights, dtype=np.float64)
102         probs = weights / weights.sum()
103         next_node = rng.choice(neighbors, p=probs)
104         walk.append(int(next_node))
105     return walk
106
107
108 def generate_walks(adj, num_walks, walk_length, p, q, seed
109 =42, num_workers=None):
110     """Generate Node2Vec walks sequentially (lower overhead
111     on macOS)."""
112     rng = np.random.default_rng(seed)
113     nodes = np.arange(len(adj))
114     walks = []
115
116     print("Generating walks (sequential - optimal for macOS)
117     ...")
118     total_walks_generated = 0
119
120     for walk_num in range(num_walks):
121         rng.shuffle(nodes)
122         print(f" Walk batch {walk_num + 1}/{num_walks}: ",
123               end="")
124
125         for node in nodes:
126             if adj[node]:
127                 walk = node2vec_walk(int(node), walk_length,
128                                     adj, p, q, rng)
129                 walks.append(walk)
130                 total_walks_generated += 1
131
132         print(f" ({total_walks_generated} total)")
133
134     return walks
135
136
137 def build_pairs(walks, window_size):
138     pairs = []
139     for walk in walks:
140         for i, center in enumerate(walk):
141             left = max(0, i - window_size)
142             right = min(len(walk), i + window_size + 1)
143             for j in range(left, right):

```

```

139         if j == i:
140             continue
141         pairs.append((center, walk[j]))
142     return pairs
143
144
145 def train_skipgram(pairs, num_nodes, embed_dim, neg_dist,
146                   num_neg=5, epochs=3, batch_size=2048, lr=0.01):
147     # Use GPU (MPS on macOS, CUDA on Linux/Windows) if
148     available
149     if torch.backends.mps.is_available():
150         device = torch.device("mps")
151         print("Using Metal Performance Shaders (GPU
152               acceleration)")
153     elif torch.cuda.is_available():
154         device = torch.device("cuda")
155         print("Using CUDA (GPU acceleration)")
156     else:
157         device = torch.device("cpu")
158         print("Using CPU")
159
160     emb_in = nn.Embedding(num_nodes, embed_dim).to(device)
161     emb_out = nn.Embedding(num_nodes, embed_dim).to(device)
162     nn.init.uniform_(emb_in.weight, -0.5 / embed_dim, 0.5 /
163                     embed_dim)
164     nn.init.zeros_(emb_out.weight)
165
166     optimizer = optim.Adam([*emb_in.parameters(), *emb_out.
167                             parameters()], lr=lr)
168     pairs = np.asarray(pairs, dtype=np.int64)
169
170     # Pre-load pairs to GPU for better performance
171     pairs_gpu = torch.tensor(pairs, dtype=torch.long, device
172                             =device)
173     neg_dist = torch.tensor(neg_dist, dtype=torch.float32,
174                             device=device)
175
176     for epoch in range(epochs):
177         # Shuffle on GPU
178         perm = torch.randperm(len(pairs_gpu), device=device)
179         pairs_shuffled = pairs_gpu[perm]
180
181         total_loss = 0.0
182         num_batches = (len(pairs_shuffled) + batch_size - 1)
183                     // batch_size
184
185         for batch_idx in range(num_batches):
186             start = batch_idx * batch_size
187             end = min(start + batch_size, len(pairs_shuffled
188             ))

```

```

180         batch = pairs_shuffled[start:end]
181         centers = batch[:, 0]
182         contexts = batch[:, 1]
183
184
185         pos_score = (emb_in(centers) * emb_out(contexts)
186                     ).sum(dim=1)
187         pos_loss = F.logsigmoid(pos_score).mean()
188
189         neg_samples = torch.multinomial(neg_dist, len(
190             batch) * num_neg, replacement=True)
191         neg_samples = neg_samples.view(len(batch),
192             num_neg)
193         neg_emb = emb_out(neg_samples)
194         center_emb = emb_in(centers).unsqueeze(2)
195         neg_score = torch.bmm(neg_emb, center_emb).
196             squeeze(2)
197         neg_loss = F.logsigmoid(-neg_score).mean()
198
199         loss = -(pos_loss + neg_loss)
200         optimizer.zero_grad()
201         loss.backward()
202         optimizer.step()
203         total_loss += loss.item() * len(batch)
204
205         if (batch_idx + 1) % max(1, num_batches // 10)
206             == 0 or batch_idx + 1 == num_batches:
207             print(f" Epoch {epoch + 1}/{epochs} - Batch
208                 {batch_idx + 1}/{num_batches}")
209
210         avg_loss = total_loss / len(pairs_shuffled)
211         print(f"Epoch {epoch + 1}/{epochs} - loss: {avg_loss
212             :.4f}")
213
214     return emb_in.weight.detach().cpu().numpy()
215
216
217 def embed_documents(tokenized_texts, vocab, embeddings):
218     dim = embeddings.shape[1]
219     doc_vecs = np.zeros((len(tokenized_texts), dim), dtype=
220         np.float32)
221     for i, tokens in enumerate(tokenized_texts):
222         ids = [vocab[t] for t in tokens if t in vocab]
223         if not ids:
224             continue
225         doc_vecs[i] = embeddings[ids].mean(axis=0)
226     return doc_vecs
227
228 def plot_tsne(embeddings, vocab, freqs, out_path):

```

```

222 idx_to_word = {idx: word for word, idx in vocab.items()}
223 top_indices = np.argsort(freqs)[-200:]
224 words = [idx_to_word[i] for i in top_indices]
225
226 pos_words = {"good", "great", "excellent", "amazing", "
227             love", "wonderful", "enjoy"}
228 neg_words = {"bad", "worst", "awful", "terrible", "
229             boring", "hate", "poor"}
230
231 colors = []
232 labels = []
233 for w in words:
234     if w in pos_words:
235         colors.append("#2ca02c")
236         labels.append(w)
237     elif w in neg_words:
238         colors.append("#d62728")
239         labels.append(w)
240     else:
241         colors.append("#7f7f7f")
242         labels.append("")
243
244 coords = TSNE(n_components=2, init="pca", perplexity=30,
245             random_state=42, learning_rate="auto").fit_transform(
246             embeddings[top_indices]
247         )
248
249 plt.figure(figsize=(6, 5))
250 plt.scatter(coords[:, 0], coords[:, 1], c=colors, s=12,
251             alpha=0.8)
252 for i, label in enumerate(labels):
253     if label:
254         plt.text(coords[i, 0], coords[i, 1], label,
255             fontsize=8)
256 plt.title("t-SNE of word embeddings (top 200 words)")
257 plt.tight_layout()
258 plt.savefig(out_path, dpi=300)
259 plt.close()
260
261 def main():
262     base_dir = Path("data/aclImdb")
263     if not base_dir.exists():
264         raise FileNotFoundError("Missing data/aclImdb.
265             Download and extract the IMDb dataset.")
266
267     max_train = 2000
268     max_test = 1000
269     vocab_size = 5000

```



```

265     window_size = 2
266
267     print("Loading dataset...")
268     train_texts, train_labels = load_imdb_split(base_dir, "
269         train", max_per_class=max_train)
270     test_texts, test_labels = load_imdb_split(base_dir, "
271         test", max_per_class=max_test)
272
273     stopwords = set(ENGLISH_STOP_WORDS)
274     print("Preprocessing...")
275     train_tokens = [preprocess(text, stopwords) for text in
276         train_texts]
277     test_tokens = [preprocess(text, stopwords) for text in
278         test_texts]
279
280     vocab, freqs = build_vocab(train_tokens, vocab_size)
281     print(f"Vocab size: {len(vocab)}")
282
283     print("Building co-occurrence graph...")
284     adj = build_cooc_graph(train_tokens, vocab, window_size=
285         window_size)
286
287     print("Generating Node2Vec walks...")
288     walks = generate_walks(adj, num_walks=20, walk_length
289         =10, p=1.0, q=0.5, seed=42)
290
291     print("Building skip-gram pairs...")
292     pairs = build_pairs(walks, window_size=2)
293     print(f"Pairs: {len(pairs)}")
294
295     print("Training skip-gram...")
296     neg_dist = freqs ** 0.75
297     neg_dist = neg_dist / neg_dist.sum()
298     embeddings = train_skipgram(
299         pairs,
300         num_nodes=len(vocab),
301         embed_dim=64,
302         neg_dist=neg_dist,
303         num_neg=5,
304         epochs=6,
305         batch_size=2048,
306         lr=0.01,
307     )
308
309     print("Embedding documents...")
310     X_train = embed_documents(train_tokens, vocab,
311         embeddings)
312     X_test = embed_documents(test_tokens, vocab, embeddings)
313
314     print("Training classifier...")

```

```
308     clf = LogisticRegression(max_iter=1000)
309     clf.fit(X_train, train_labels)
310     acc = clf.score(X_test, test_labels)
311     print(f"Test accuracy: {acc:.4f}")
312
313     print("Plotting t-SNE...")
314     plot_tsne(embeddings, vocab, freqs, "word_tsne.png")
315
316
317 if __name__ == "__main__":
318     main()
```